

# **Advanced Algorithms and Data Structures**

## **Lecture# 06: Graph**

# Introduction

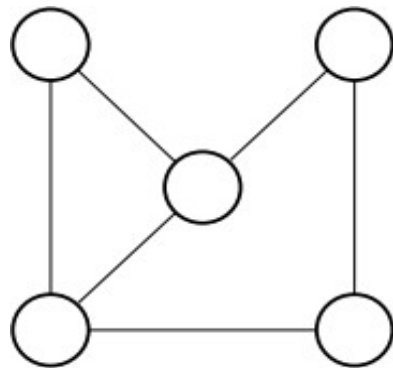
- Graphs provide the ultimate in data structure flexibility
- Here is a small sampling of the range of problems that graphs are routinely applied to:
  - Modeling connectivity in computer and communications networks.
  - Representing a map as a set of locations with distances between locations; used to compute shortest routes between locations.
  - Modeling flow capacities in transportation networks.
  - Finding a path from a starting condition to a goal condition; for example, in artificial intelligence problem solving.
  - Modeling computer algorithms, showing transitions from one program state to another.
  - Finding an acceptable order for finishing subtasks in a complex activity, such as constructing large buildings.
  - Modeling relationships such as family trees, business or military organizations, and scientific taxonomies.

# Terminologies

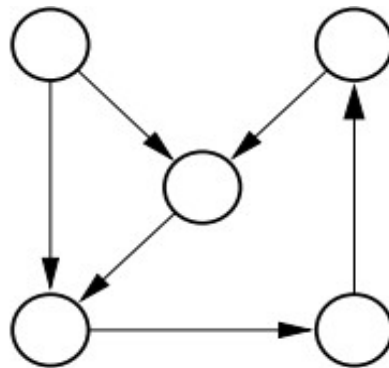
- A **graph**  $G = (V, E)$  consists of a set of vertices  $V$  and a set of edges  $E$ , such that each edge in  $E$  is a connection between a pair of vertices in  $V$
- The number of vertices is written  $|V|$ , and the number of edges is written  $|E|$
- $|E|$  can range from zero to a maximum of  $V^2 - |V|$
- A graph with relatively few edges is called **sparse**, while a graph with many edges is called **dense**
- A graph containing all possible edges is said to be **complete**

# Terminologies

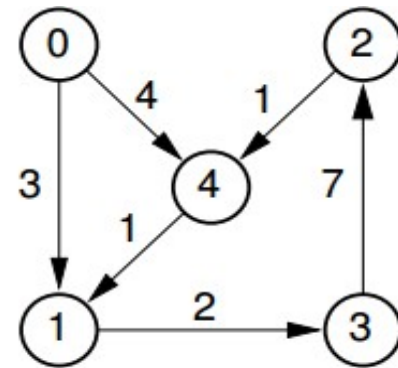
- A graph whose edges are not directed is called an **undirected graph** (a)
- A graph with edges directed from one vertex to another is called a **directed graph** or **digraph** (b)
- A graph with labels associated with its vertices is called a **labeled graph** (c)



(a)



(b)



(c)

# Terminologies

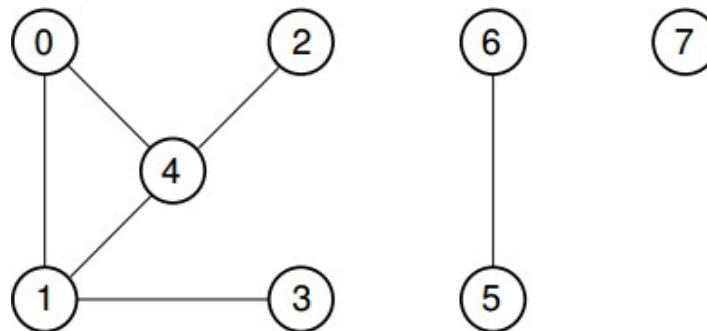
- Two vertices are **adjacent** if they are joined by an edge (they are also called neighbors)
- An edge connecting Vertices  $U$  and  $V$  is written  $(U, V)$  (such an edge is said to be **incident** on Vertices  $U$  and  $V$ )
- Associated with each edge may be a **cost** or **weight**
- Graphs whose edges have weights are said to be **weighted**

# Terminologies

- A sequence of vertices  $v_1, v_2, \dots, v_n$  forms a **path** of length  $n - 1$  if there exist edges from  $v_i$  to  $v_{i+1}$  for  $1 \leq i < n$
- A path is **simple** if all vertices on the path are distinct
- The **length** of a path is the number of edges it contains
- A **cycle** is a path of length three or more that connects some vertex  $v$  to itself
- A cycle is **simple** if the path is simple, except for the first and last vertices being the same

# Terminologies

- A **subgraph**  $S$  is formed from graph  $G$  by selecting a subset  $V_s$  of  $G$ 's vertices and a subset  $E_s$  of  $G$ 's edges such that for every edge  $E$  in  $E_s$ , both of  $E$ 's vertices are in  $V_s$
- An undirected graph is **connected** if there is at least one path from any vertex to any other
- The maximally connected subgraphs of an undirected graph are called **connected components**



# Terminologies

- A graph without cycles is called **acyclic**
- A directed graph without cycles is called a **directed acyclic graph** or **DAG**
- A **free tree** is a connected, undirected graph with no simple cycles (an equivalent definition is that a free tree is connected and has  $V - 1$  edges)

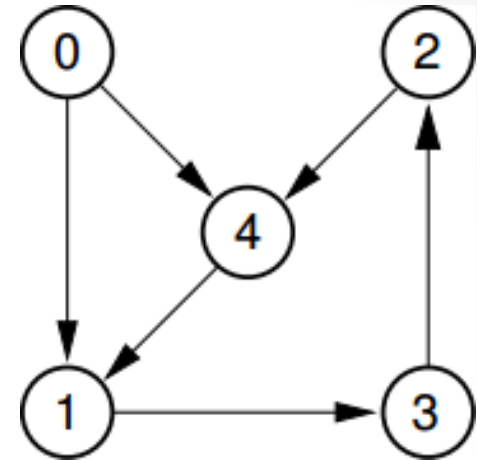


# Representations

- There are two commonly used methods for representing graphs:
  - adjacency matrix
  - adjacency list
- Both representations can be used to store directed or undirected graphs

# Adjacency Matrix

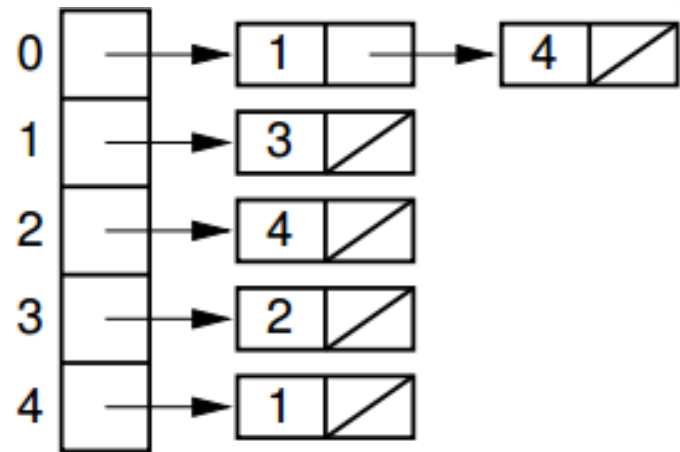
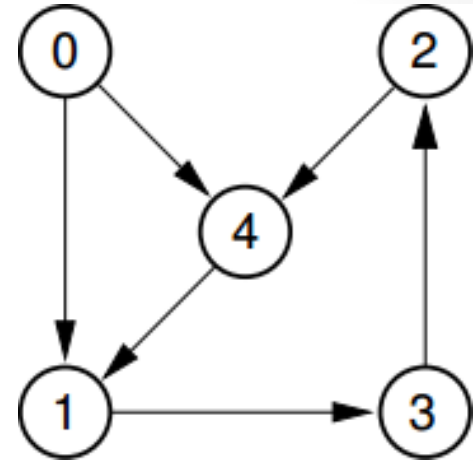
- The adjacency matrix for a graph is a  $V \times |V|$  array
- Assume that  $V = n$  and that the vertices are labeled from  $v_0$  through  $v_{n-1}$ 
  - Row  $i$  of the adjacency matrix contains entries for Vertex  $v_i$
  - Column  $j$  in row  $i$  is marked if there is an edge from  $v_i$  to  $v_j$  and is not marked otherwise
  - If we wish to associate a number with each edge, such as the weight, then each matrix position must store that number



|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 |   | 1 |   |   | 1 |
| 1 |   |   |   | 1 |   |
| 2 |   |   |   |   | 1 |
| 3 |   |   | 1 |   |   |
| 4 |   | 1 |   |   |   |

# Adjacency Matrix

- The adjacency list is an array of linked lists
- The array is  $|V|$  items long, with position  $i$  storing a pointer to the linked list of edges for Vertex  $v_i$



# Graph Implementations

- See `Graph.java` for graph ADT
- Vertices are defined by an integer index value (0, 1, 2, ...)
- We can assume that a graph application stores any additional information of interest about a given vertex elsewhere (that is why we do not use a generic)
- Abstract class `Graph` has methods to return the number of vertices and edges (methods `n` and `e`, respectively)
- Function `weight` returns the weight of a given edge (it returns 0 if no such edge exists)
- Functions `setEdge` and `delEdge` set the weight of an edge and remove an edge from the graph, respectively

# Graph Implementations

- Function `first` takes as input a vertex  $v$ , and returns the edge to the first neighbor for  $v$  (we assume the neighbor list is sorted by vertex number)
- Function `next` takes as input vertices  $v_1$  and  $v_2$  and returns the index for the vertex forming the next edge with
  - $v_1$  after  $v_2$  on  $v_1$ 's edge list
- Function `next` will return a value of  $n = |V|$  once the end of the edge list for  $v_1$  has been reached
- The implementations using both adjacency matrix and list can be found in `Graphm.java` and `Graphl.java` respectively

# Graph Traversals

- Graph traversal algorithms typically begin with a start vertex and attempt to visit the remaining vertices from there
- Graph traversals must deal with a number of troublesome cases:
  - It may not be possible to reach all vertices from the start vertex when the graph is not connected
  - The graph may contain cycles which may cause the algorithm to go into an infinite loop
- Solution:
  - Maintaining a mark bit for each vertex
  - If a marked vertex is encountered during traversal, it is not visited a second time
  - If not all vertices are marked after the traversal completes, we can continue the traversal from another unmarked vertex

# Graph Traversals

- To ensure visiting all vertices, `graphTraverse` could be called as follows on a graph  $G$ :

```
void graphTraverse(Graph G) {  
    int v;  
    for (v=0; v<G.n(); v++)  
        // Initialize  
        G.setMark(v, UNVISITED);  
    for (v=0; v<G.n(); v++)  
        if (G.getMark(v) == UNVISITED)  
            doTraverse(G, v);  
}
```

- Function `doTraverse` might be implemented by using one of the graph traversals described next

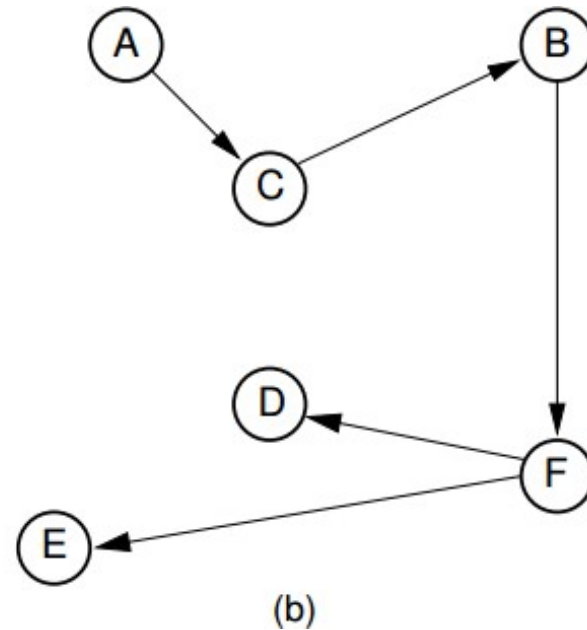
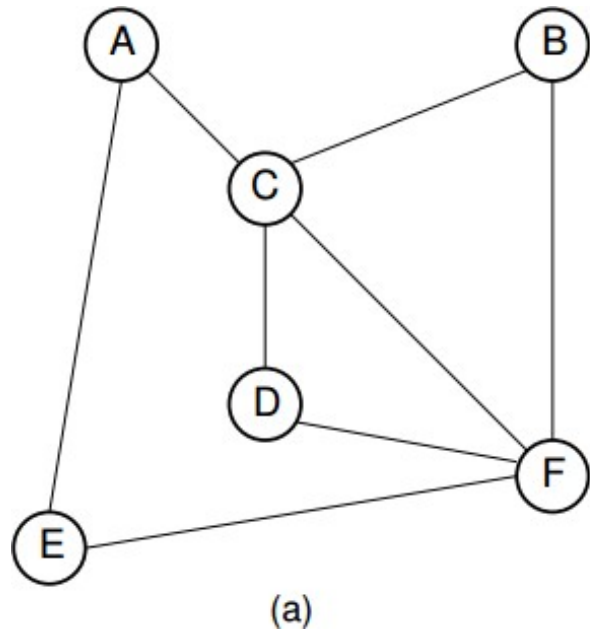
# Depth-First Search (DFS)

- Whenever a vertex  $v$  is visited during the search, DFS will recursively visit all of  $v$ 's unvisited neighbors

```
/** Depth first search */
static void DFS(Graph G, int v) {
    // Take appropriate action
    PreVisit(G, v);
    G.setMark(v, VISITED);
    for (int w = G.first(v); w < G.n(); w =
G.next(v, w))
        if (G.getMark(w) == UNVISITED)
            DFS(G, w);
    // Take appropriate action
    PostVisit(G, v);
}
```



# Depth-First Search (DFS)

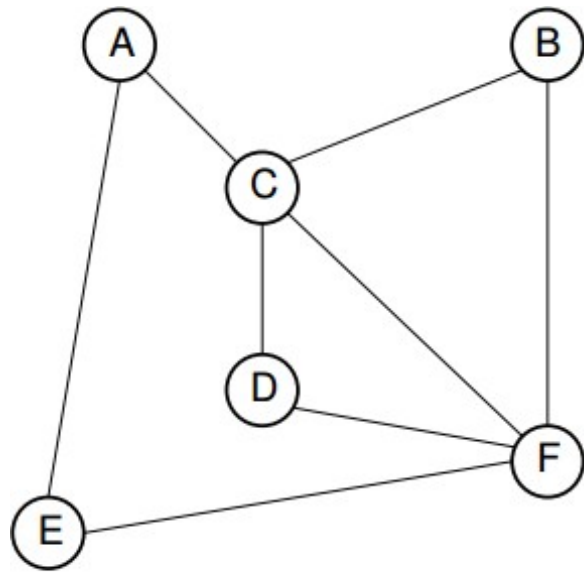


# Breadth-First Search (BFS)

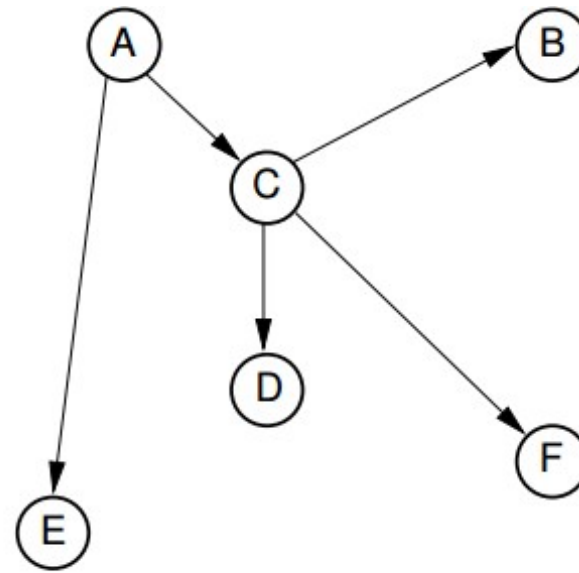
- BFS examines all vertices connected to the start vertex before visiting vertices further away

```
/** Breadth first (queue-based) search */
static void BFS(Graph G, int start) {
    Queue<Integer> Q = new ArrayQueue<Integer>(G.n());
    Q.enqueue(start);
    G.setMark(start, VISITED);
    // Process each vertex on Q
    while (Q.length() > 0) {
        int v = Q.dequeue();
        // Take appropriate action
        PreVisit(G, v);
        for (int w = G.first(v); w < G.n(); w = G.next(v, w))
            // Put neighbors on Q
            if (G.getMark(w) == UNVISITED) {
                G.setMark(w, VISITED);
                Q.enqueue(w);
            }
        // Take appropriate action
        PostVisit(G, v);
    }
}
```

# Breadth-First Search (BFS)



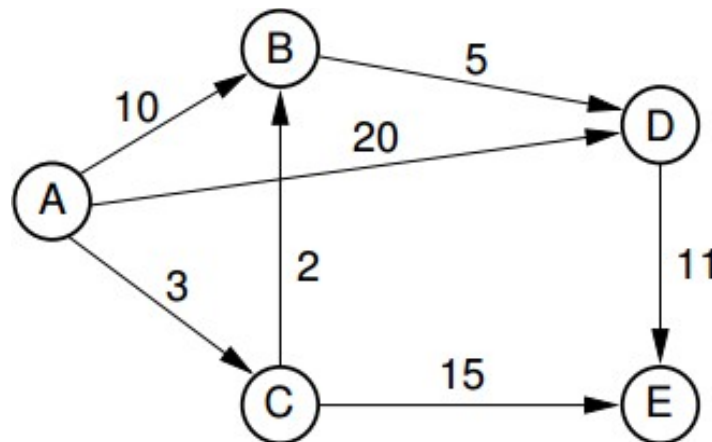
(a)



(b)

# Shortest-Path Problems

- Given such a graph, a typical problem is to find the total length of the shortest path between two specified vertices
- We define  $w(v_1, v_2)$  to be the weight of edge  $(v_1, v_2)$   
 $w(v_1, v_2) = \infty$  if there is no edge  $(v_1, v_2)$
- We use notation  $d(v_1, v_2)$  to indicate the shortest path from  $v_1$  to  $v_2$   
 $d(v_1, v_2) = \infty$  if there is no path from  $v_1$  to  $v_2$



# Dijkstra's Algorithm

- It works by maintaining a distance estimate  $D(v)$  for all vertices  $v$  in  $V$
- A listing for progress of Dijkstra's algorithm operating on the graph in the previous slide

|           | <b>A</b> | <b>B</b> | <b>C</b> | <b>D</b> | <b>E</b> |
|-----------|----------|----------|----------|----------|----------|
| Initial   | 0        | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| Process A | 0        | 10       | 3        | 20       | $\infty$ |
| Process C | 0        | 5        | 3        | 20       | 18       |
| Process B | 0        | 5        | 3        | 10       | 18       |
| Process D | 0        | 5        | 3        | 10       | 18       |
| Process E | 0        | 5        | 3        | 10       | 18       |

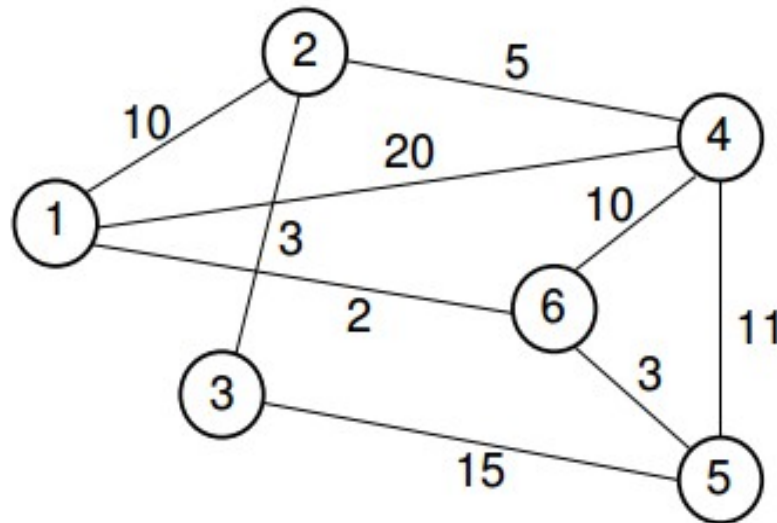
# Dijkstra's Algorithm

```
// Compute shortest path distances from s, store them in D static
void Dijkstra(Graph G, int s, int[] D) {
    for (int i=0; i<G.n(); i++)
        // Initialize
        D[i] = Integer.MAXVALUE;
        D[s] = 0;
    for (int i=0; i<G.n(); i++) { // Process the vertices
        int v = minVertex(G, D); // Find next-closest vertex
        G.setMark(v, VISITED);
        if (D[v] == Integer.MAX VALUE) return; // Unreachable
        for (int w = G.first(v); w < G.n(); w = G.next(v, w))
            if (D[w] > (D[v] + G.weight(v, w)))
                D[w] = D[v] + G.weight(v, w);
    }
}

static int minVertex(Graph G, int[] D) {
    int v = 0; // Initialize v to any unvisited vertex;
    for (int i=0; i<G.n(); i++)
        if (G.getMark(i) == UNVISITED) { v = i; break; }
    for (int i=0; i<G.n(); i++) // Now find smallest value
        if ((G.getMark(i) == UNVISITED) && (D[i] < D[v]))
            v = i;
    return v;
}
```

# Exercises

1. Draw the adjacency matrix and list for this graph.
2. If a pointer requires four bytes, a vertex label requires two bytes, and an edge weight requires two bytes, which representation requires more space for this graph?
3. Show the DFS and BFS tree for this graph starting at vertex 1.
4. Calculate the shortest paths generated by running Dijkstra's shortest-paths algorithm on this graph beginning at vertex 4.



# References

- Clifford A. Shaffer. (2012). *Data Structures and Algorithm Analysis*. Department of Computer Science. Virginia Tech.
- Douglas Wilhelm Harder. (2013). Queues. University of Waterloo, Waterloo, Ontario.
- Kurt Mehlhorn and Peter Sanders. (2007). *Algorithms and Data Structures*. Springer.
- Naveen Garg. (2008). *Data Structures and Algorithms*. Department of Computer Science and Engineering. Indian Institute of Technology, Delhi.
- Robert Lafore. (2003). *Data Structures & Algorithms in Java*. Sams Publishing.